

A PROJECT REPORT
ON
OBJECT DETECTION AND DEPTH ESTIMATION
USING REALSENSE D435i CAMERA

SUBMITTED BY

Mariam Thomas – 22070127037

Saachi Sharma – 22070127057

Sairaj Pisal - 23070127504



UNDER THE GUIDANCE OF

Dr. Manoj Kumar Shukla

Assistant Professor

Department of Robotics and Automation

SYMBIOSIS INSTITUTE OF TECHNOLOGY

A Constituent of Symbiosis International (Deemed University)

Pune - 412115

CONTENTS

ABSTRACT	2
I. INTRODUCTION	3
1.1 Intel® RealSense™ Depth Camera D435i	3
1.2 Robot Operating System (ROS)	4
1.3 ROS Visualization (Rviz)	5
1.4 Open-Source Computer Vision Library (OpenCV)	6
1.5 PointCloud	6
1.5 Orientation and Coordinate Systems	7
II. LITERATURE REVIEW	8
2.1 Object Detection in Robotic Systems	8
2.2 Vision System for Service Robots	8
2.3 Integrating 2D LiDAR, RGB-D Camera for Autonomous Mobile Robot Navigation ...	8
III. STATEMENT OF PROBLEM	9
IV. OBJECTIVES	10
V. METHODOLOGY	10
5.1 Hardware Setup	10
5.2 Connecting the RealSense Camera and Visualizing PointCloud in RViz2	11
5.3 Creating Custom ROS 2 Package and Python Script	12
5.4 Writing the Launch File and Running the Node	13
5.5 Running All Nodes and Launching YOLO Detection with Depth Visualization	14
VI. RESULT AND ANALYSIS	15
VII. CONCLUSION AND FUTURE SCOPE	17
REFERENCES	18

ABSTRACT

This project demonstrates the development and integration of a real-time object detection and depth estimation system using the Intel RealSense D435i RGB-D camera with ROS 2 (Humble), OpenCV, and YOLOv8 object detection model. The objective was to create a low-cost, modular system that can detect multiple objects within the field of view of the camera and estimate their distances based on aligned depth information.

The RealSense camera streams RGB and depth data, which is handled by a special ROS 2 Python node doing detection and adding the results as bounding boxes and distance labels. RViz2 and PointCloud2 are utilized for 3D visualization of the scene and sensor data.

The system was tested indoors and proved to exhibit stable real-time performance, good depth estimation accuracy, and solid object identification reliability. The findings confirm the system's usability for robotic navigation, environment mapping, and spatial awareness applications.

I. INTRODUCTION

In the rapidly evolving field of robotics and computer vision, the ability to accurately detect objects and estimate their depth in real-time is crucial for applications such as autonomous navigation, manipulation, and environment mapping. The Intel RealSense D435i depth camera, equipped with both RGB and depth sensors, has become a popular choice for developers aiming to achieve these capabilities. When integrated with Robot Operating System 2 (ROS2), OpenCV, and RViz, the D435i offers a comprehensive platform for developing sophisticated perception systems.

1.1 INTEL REALSENSE DEPTH CAMERA D435I

Intel® RealSense™ Depth Camera D435i is a sophisticated depth-sensing camera that comes with an Inertial Measurement Unit (IMU) to offer superior spatial and motion information. This feature allows developers to design applications needing real-time motion tracking and depth perception, including robotics, augmented reality, and 3D scanning.

Including an IMU enables the application to increase its depth sensitivity in any environment where the camera is in motion. This opens the potential for basic SLAM and tracking applications enabling enhanced point-cloud alignment. It enables enhanced environmental perception for robotics and drones as well. Utilization of an IMU simplifies registration and calibration for handheld scanning system applications and is also significant in areas like virtual/augmented reality and drones.

Inertial measurement unit is employed in the detection of rotations and movements in 6 degrees of freedom (6DoF). An IMU is a combination of a range of sensors and gyroscopes to identify rotation and movement in 3 axes, and pitch, yaw and roll. It is applied in gaming and pointing devices, as well as image stabilization.

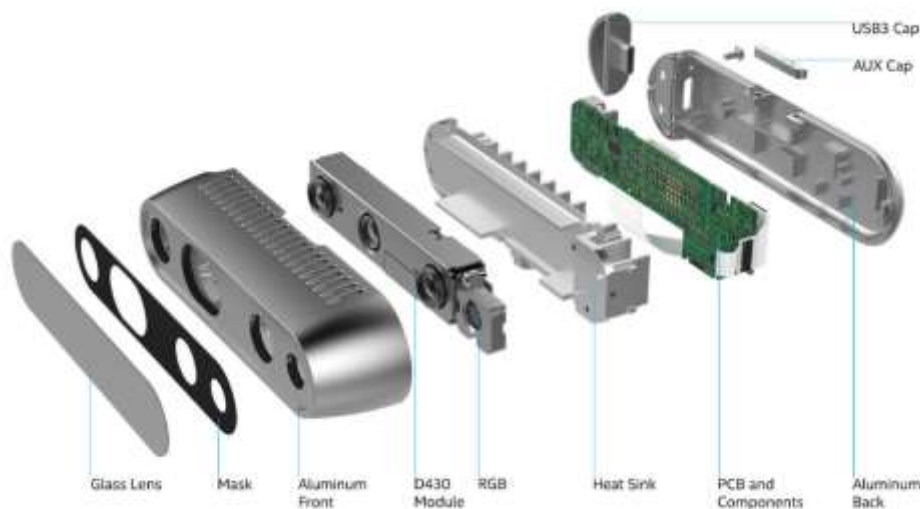


Fig 1. Intel® RealSense™ Depth Camera D435i

1.1.1 Specifications

- Depth Technology - Active Stereoscopic
- Operating Range (Min-Max) - ~ .3m - 3m
- Depth Resolution and FPS - 1280x720, 30fps
- Depth Field of View - H:87 V:58
- RGB Sensor - Yes
- Tracking Module - No
- Dimensions - 90mx25mx25m (Camera)
- System Interface Type - USB 3

1.1.2 Applications of Intel® RealSense™ Depth Camera D435i

1. Robot Navigation and Obstacle Detection - In robotics, the D435i facilitates better navigation by giving real-time depth information, which allows robots to detect and navigate around obstacles more efficiently. Its global shutter sensors and wide field of view ensure that it can be used in dynamic environments.
2. 3D Scanning and Modelling - The D435i enables 3D scanning through the capture of high-resolution depth information, enabling the generation of precise 3D models. This feature is useful in applications such as digital content creation and industrial design.
3. Virtual Reality (VR) and Augmented Reality (AR) - In VR and AR, the D435i is used in creating immersive experiences through the accurate depth data provided, allowing virtual objects to have realistic interaction with the real environment.
4. Agricultural Monitoring - The D435i is utilized in agricultural applications for monitoring the health of crops and plant phenotyping through the use of depth sensing to determine plant structures.
5. Security and Surveillance - The camera provides improvement to security systems by providing depth information that supports precise motion detection and object identification, enhancing monitoring accuracy.

1.2 ROBOT OPERATING SYSTEM

Robot Operating System 2, is an open-source framework that helps developers build robot applications in a modular and scalable way. While the name might suggest it's an operating system, it is a collection of tools, libraries, and conventions that work *with* your existing operating system to control robot behaviour. For this project where we are working with the Intel RealSense D435i camera to perform object detection and depth estimation—ROS 2 is the backbone that keeps everything connected and communicating efficiently.

1.2.1 Features of ROS 2

Improved Communication: ROS 2 uses DDS (Data Distribution Service), which allows for real-time, secure, and reliable communication between different components. This is perfect for handling live camera streams and sensor data.

- Better Performance and Reliability: Humble gives us better support for real-time systems—useful when we need to process depth and image data without delays.
- Multi-platform Support: ROS 2 works well across different platforms and operating systems, making development and deployment much easier.
- Scalability: Whether we're working on a single robot or multiple devices, ROS 2 makes it easy to scale and manage communication across components.

1.3 RVIZ

RViz (ROS Visualization) is a 3D visualization tool within the Robot Operating System framework. It enables developers to visualize a robot's sensor data, state, and environment, facilitating the interpretation and debugging of complex robotic systems.

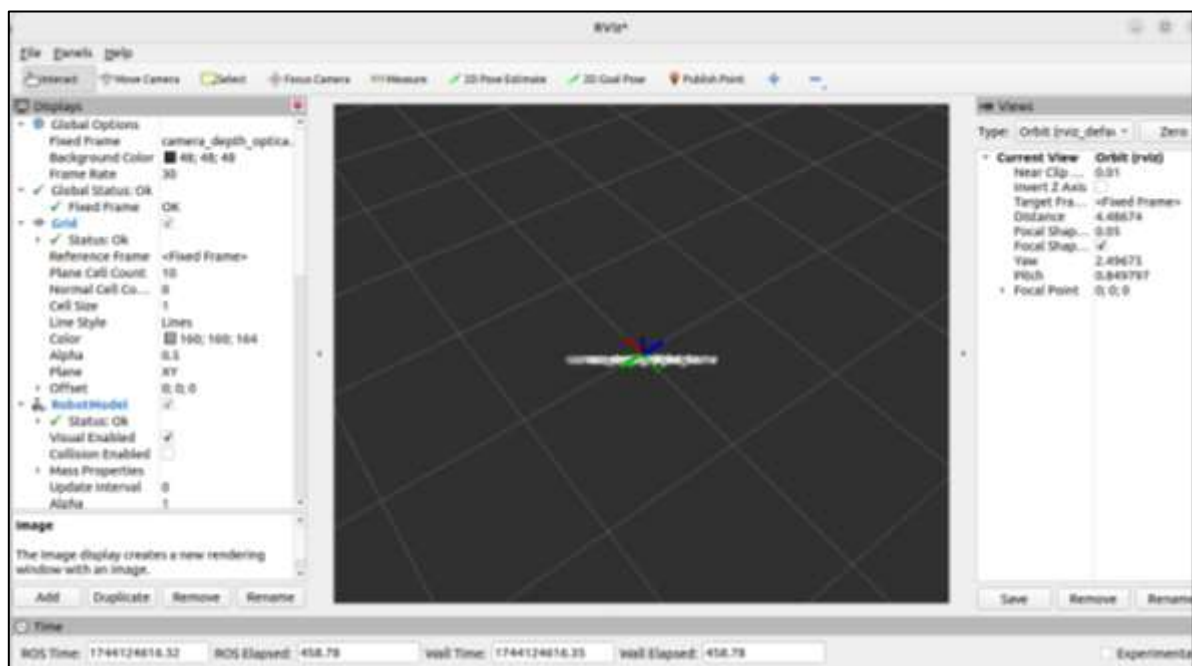


Fig 2. ROS Visualization

RViz allows us to visualize the depth and colour data streams from the D435i camera in real-time. This immediate feedback is essential for verifying that the camera captures accurate data and that our object detection algorithms function correctly. By visualizing the robot's perception of its environment, we can identify and troubleshoot issues in the data processing pipeline. Object detection and depth estimation inherently involve understanding spatial relationships.

RViz provides a 3D representation of the environment, making it easier to comprehend how detected objects are positioned relative to the camera and each other.

1.4 OPENCV

OpenCV (Open-Source Computer Vision Library) is an open-source computer vision and machine learning library designed to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in commercial products. Developed initially by Intel in 1999, OpenCV has become a widely adopted library in both academic and industrial fields.

1.5 POINTCLOUD

A PointCloud is a collection of data points defined in a three-dimensional coordinate system, where each point represents a specific location on the surface of an object or within an environment. These points are typically defined by their X, Y, and Z coordinates, capturing the spatial dimensions of the subject. Point clouds are commonly generated using 3D scanning technologies such as LiDAR (Light Detection and Ranging), photogrammetry, or other 3D imaging methods.

The Intel RealSense D435i publishes depth data via ROS2 and here this data is converted into a point cloud. We visualize it using RViz and analyze object positions with OpenCV. The point cloud helps us determine where objects are in relation to the camera (e.g., 3D bounding boxes).

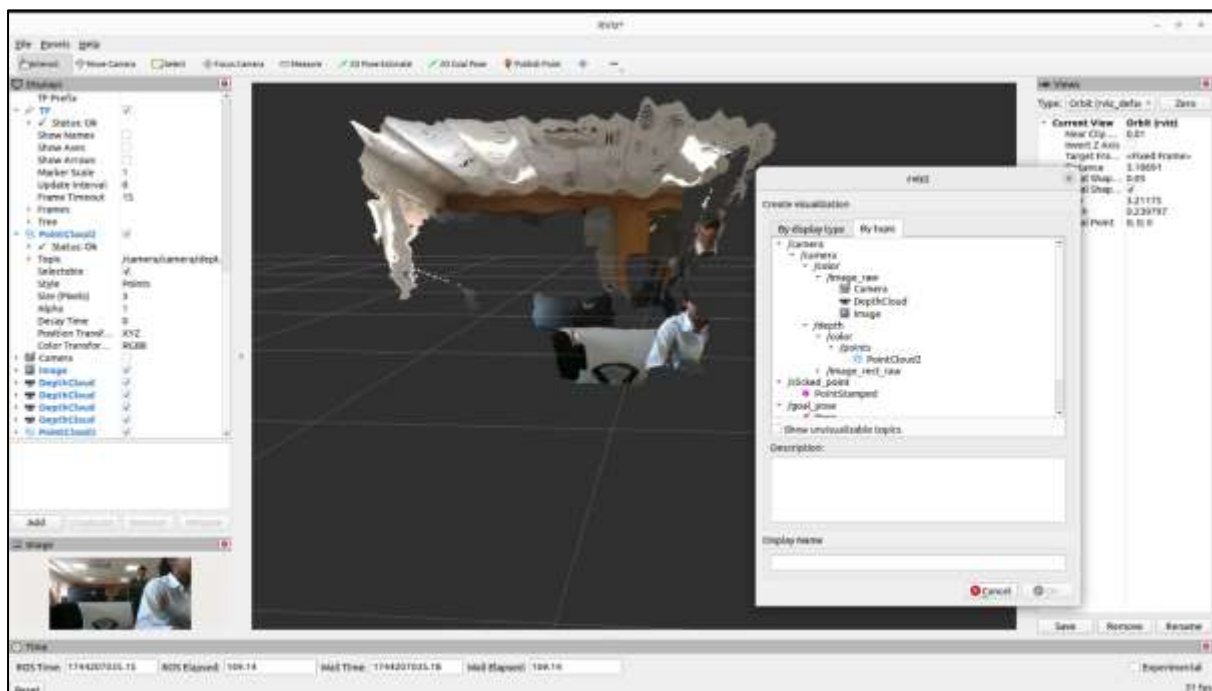


Fig 3. Point Cloud

1.5.1 Parameters of PointCloud

- X, Y, Z Coordinates – Core of any point cloud. Represents the 3D position of each point in space.
- RGB (Colour Information) - Some point clouds carry colour data for each point (Red, Green, Blue).
- Intensity - Captures the strength of the returned signal
- Normals - These are vectors that describe the orientation of a surface at each point. It is used for object recognition and surface reconstruction.
- Timestamp - Used in dynamic environments to know when each point was recorded.

1.5.2 PointCloud Library

The PointCloud Library (PCL) is a powerful and widely used open-source library for 3D point cloud processing, written entirely in C++. It includes modules for filtering, segmentation, surface reconstruction, visualization, and feature estimation. Due to the computationally intensive nature of point cloud data—often involving millions of 3D points per frame—PCL is implemented in C++ to ensure high performance and low latency.

1.6 ORIENTATION AND COORDINATE SYSTEMS

In any computer vision or robotic perception system, the concept of orientation and coordinate frames is critical to correctly interpret and represent data in three-dimensional space. This project involves multiple data streams such as RGB images, depth images, and point clouds, all of which are aligned and transformed according to standardized coordinate frames used in both ROS 2 and the Intel RealSense D435i depth camera.

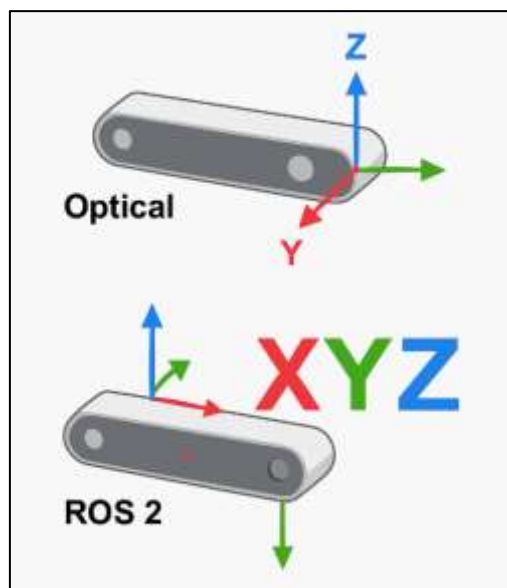


Fig 4. Comparison of Coordinate Frames: ROS 2 vs RealSense Optical Frame Orientation

1.6.1 Camera Coordinate System

The Intel RealSense D435i follows a typical optical coordinate system, defined as:

- **Z-axis:** Points **forward** from the camera (into the scene).
- **X-axis:** Points **rightward** from the center of the camera.
- **Y-axis:** Points **downward** from the camera's perspective.

This orientation is applied when depth values are extracted for each pixel in the RGB image, or when point cloud data is generated. The camera also reports pose and motion information using this coordinate convention, which is essential for depth estimation and object localization.

1.6.2 ROS 2 TF Frames and Transformation

ROS 2 uses a system called TF2 (Transform Library) to manage coordinate transformations between different reference frames. In this project, several coordinate frames are automatically published by the RealSense ROS wrapper, including:

- `/camera_link`: Reference frame for the RGB camera.
- `/camera_depth_frame`: Reference frame for the depth sensor.
- `/camera_color_frame`: Frame for the color image stream.
- `/camera_depth_optical_frame`: Optical frame used for depth visualization.
- `/camera_color_optical_frame`: Optical frame for RGB data.

These frames are essential for accurately transforming 2D image coordinates into 3D positions in space. The TF tree allows RViz and other ROS tools to align visual and spatial data, enabling precise representation of detected objects and their positions relative to the camera.

II. LITERATURE REVIEW

2.1 Object Detection in Robotic Systems

In object detection in robotic systems, several approaches have been attempted, with the focus being on accurate localization of target objects. The paper describes a robust system that employs the Scale-Invariant Feature Transform (SIFT) for feature extraction from template and test images, enabling object recognition through accurate keypoint matching. This is supplemented by the utilization of affine transformations to effectively trace the object's location within the image. The earlier research, e.g., the parts-based detection algorithm suggested by Fischler and Elschlager and geometrical structures integration for object description, points toward the necessity of merging multiple features and algorithms in improving detection accuracy. The study also builds on these developments by utilizing a robot arm equipped with the Robot Operating

System (ROS), where real-time image processing and object localization are facilitated, thus enhancing the overall effectiveness of robotic manipulation tasks in cluttered spaces.

2.2 Vision System for Service Robots

The development of a vision system for service robots that makes use of both OpenCV and ROS is discussed. ROS is central to the integration of the functionality of the vision system through enabling communication between processing algorithms and sensors of the robot. The system uses a two-camera setup, Microsoft Kinect for depth sensing and RGB object recognition and a stereo camera for controlling an articulated robotic arm to pick up objects. OpenCV offers the image processing tools required, such as edge detection and colour filtering, while ROS makes sure that data from the cameras is properly processed in real-time so that the robot can efficiently interact with its environment

2.3 Integrating 2D LiDAR and RGB-D Camera for Autonomous Mobile Robot Navigation

In their 2019 study, Gatesichapakorn et al. proposed and evaluated two ROS-based autonomous navigation systems for mobile robots operating in dynamic indoor environments. The first system employed a 2D LiDAR sensor alone, while the second incorporated both a 2D LiDAR and an RGB-D camera to overcome the depth limitations of single-plane laser scanning. The authors used the ROS navigation stack, Docker containers for deployment, and low-cost onboard computers such as the Raspberry Pi 3 and Intel NUC

Their methodology involved generating static maps using the *slam_gmapping* package and implementing obstacle avoidance using costmap layers—*ObstacleCostmapPlugin* for LiDAR and *VoxelCostmapPlugin* for RGB-D depth data. Depth data were pre-processed using PCL filters like *voxel_grid* and *StatisticalOutlierRemoval*. Performance was evaluated through waypoint-based navigation tasks in real-world and simulated environments. The RGB-D-enhanced system was particularly effective in detecting elevated or narrow obstacles that 2D LiDAR alone could miss, demonstrating improved navigation safety and robustness.

III. STATEMENT OF PROBLEM

Traditional navigation systems using only 2D LiDAR or RGB cameras struggle to detect obstacles accurately in dynamic 3D environments, leading to safety risks and limited spatial awareness. There is a need for a real-time, low-cost system that combines object detection and depth estimation to improve obstacle avoidance. This project addresses that need by integrating the Intel RealSense D435i camera with ROS2, OpenCV, and RViz to enhance navigation accuracy and environmental understanding.

IV. OBJECTIVES

1. To integrate the Intel RealSense D435i camera with ROS2 (Humble) for real-time depth and image data processing.
2. To implement object detection using OpenCV for identifying obstacles in the environment.
3. To estimate the distance of detected objects using aligned depth data from the RealSense camera.
4. To visualize the robot's environment and navigation path using RViz.

V. METHODOLOGY

This project involves building a real-time object detection and depth estimation system using the Intel RealSense D435i camera integrated with ROS 2 (Humble), OpenCV, and RViz2.

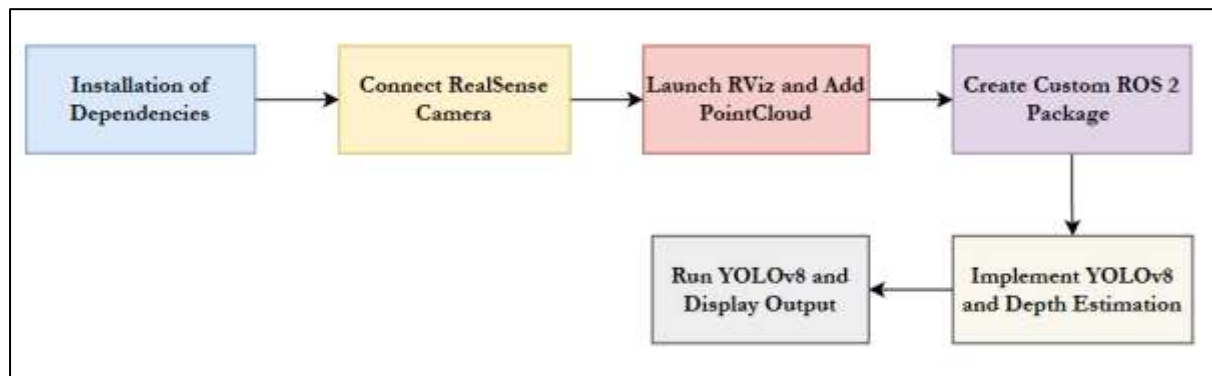


Fig 5. Methodology followed in this study

5.1 Hardware Setup

The Intel RealSense D435i camera is connected to a system running Ubuntu with ROS 2 Humble and install necessary drivers and packages: librealsense, realsense2_camera, OpenCV, and ultralytics (for YOLOv8).

Component	Specification / Tool
Camera	Intel RealSense D435i
Operating System	Ubuntu 22.04
ROS Version	ROS 2 Humble Hawksbill
Programming Language	Python 3 (reclpy)
Libraries	OpenCV, Ultralytics YOLOv8, cv_bridge
Visualization	RViz2

Fig 6. Hardware and software specifications

5.2 Connecting the RealSense Camera and Visualizing PointCloud in RViz2

After physically connecting the Intel RealSense D435i camera to the system via USB, the next step involved launching RViz2, the primary 3D visualization tool in ROS2. RViz2 was opened using the `rviz2` command in the terminal. Within the RViz2 interface, we added a new display by selecting the `/camera/depth/color/points` topic under the PointCloud2 message type.

This topic is responsible for publishing the 3D point cloud data generated by the RealSense camera. In a new terminal, the RealSense ROS2 wrapper was launched using the command which initialized the RGB and depth data streams from the camera and enabled real-time point cloud generation.

```
ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true
```

As a result, RViz2 began rendering a live 3D point cloud view of the environment, allowing us to observe the spatial structure of the surroundings in real-time. This step was crucial for validating that the camera was functioning properly, the topics were correctly published, and the system could successfully visualize depth data before proceeding to object detection and analysis.

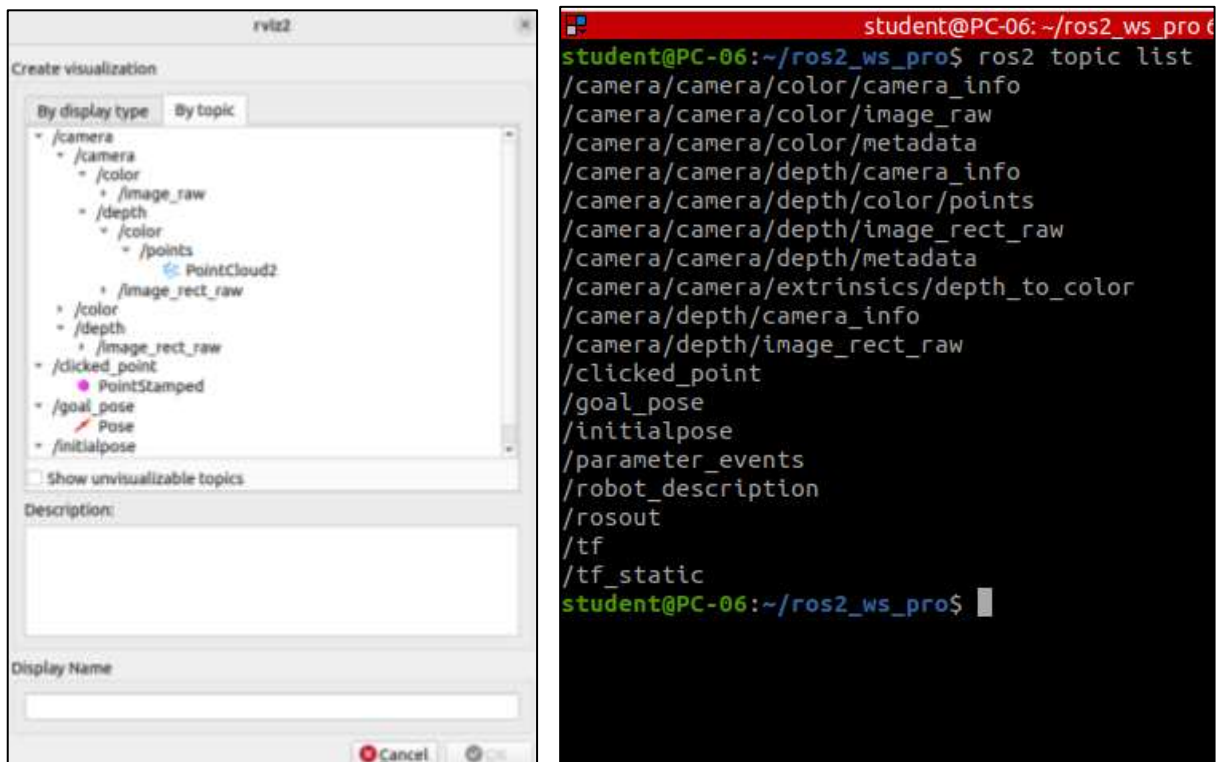


Fig 7. Adding PointCloud and checking topic list

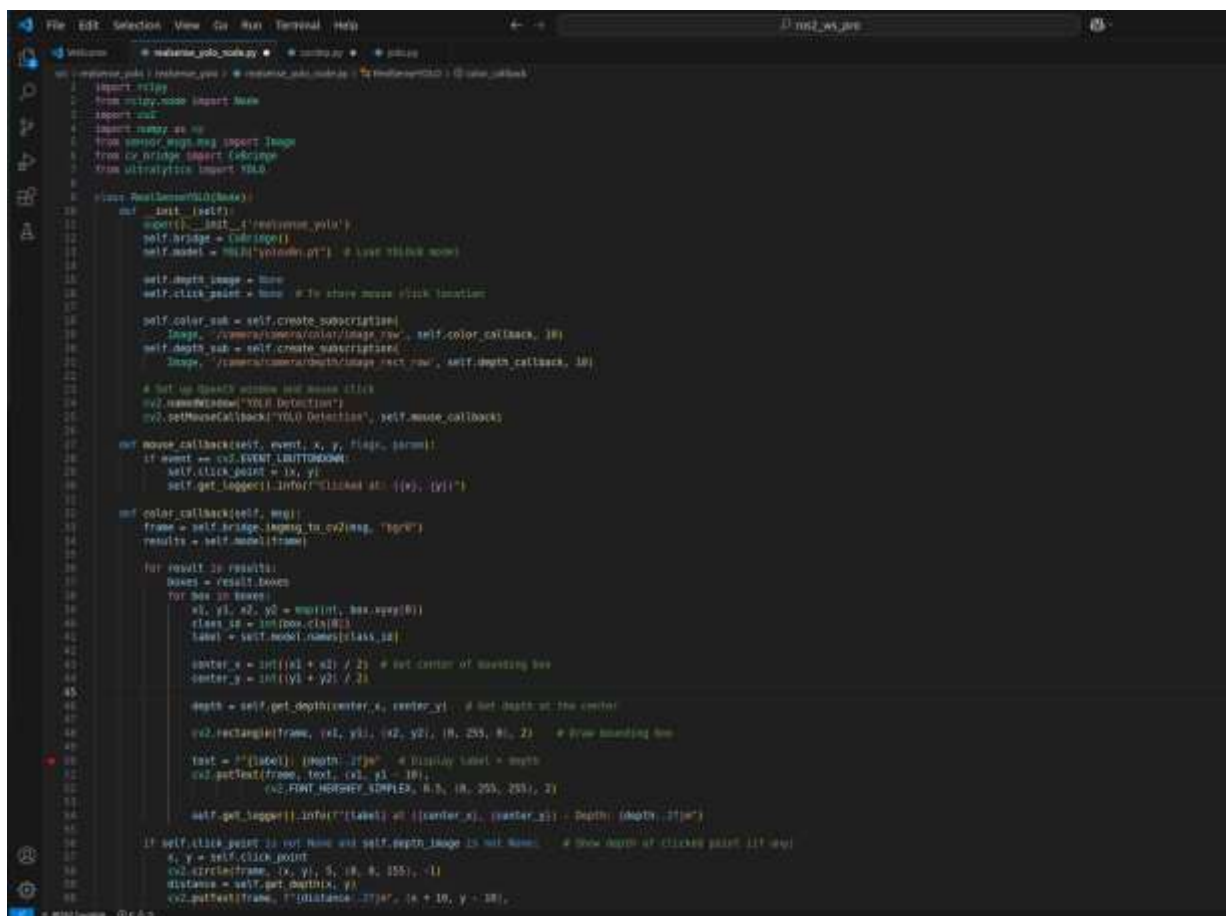
5.3 Creating Custom ROS 2 Package and Python Script

To perform object detection and depth estimation using the RealSense D435i camera, a custom ROS 2 package named `realsense_yolo` was created. This package was built using Python with `rcipy` as the client library. The package was initialized using the command

```
ros2 pkg create realsense_yolo --build-type ament_python --dependencies rcipy sensor_msgs cv_bridge
```

The next step was to implement the main functionality by writing a Python script named `realsense_yolo_node.py`.

It integrates the YOLOv8 object detection model with the depth data from the RealSense D435i camera in a ROS 2 node. Its main purpose is to detect objects in real-time from the RGB camera feed and estimate their distance using aligned depth data. The output is a visual stream where each detected object is shown with a bounding box, label (like "person", "tv"), and its real-world distance from the camera (e.g., "person: 1.20 m").



```
1 #!/usr/bin/env python3
2 import rcipy
3 from rcipy.node import Node
4 import cv2
5 import numpy as np
6 from sensor_msgs.msg import Image
7 from cv_bridge import CvBridge
8 from ultralytics import YOLO
9
10 class RealSenseYOLO(Node):
11     def __init__(self):
12         super().__init__('realsense_yolo')
13         self.bridge = CvBridge()
14         self.model = YOLO('yolov8.pt') # Load YOLO model
15
16         self.depth_image = None
17         self.click_point = None # To store mouse click location
18
19         self.color_sub = self.create_subscription(
20             Image, '/camera/color/image_raw', self.color_callback, 10)
21         self.depth_sub = self.create_subscription(
22             Image, '/camera/depth/image_rect_raw', self.depth_callback, 10)
23
24         # Set up ROS2 window and mouse click
25         cv2.namedWindow("YOLO Detection")
26         cv2.setMouseCallback("YOLO Detection", self.mouse_callback)
27
28     def mouse_callback(self, event, x, y, flags, param):
29         if event == cv2.EVENT_LBUTTONDOWN:
30             self.click_point = (x, y)
31             self.get_logger().info("Clicked at: (%s, %s)" % (x, y))
32
33     def color_callback(self, msg):
34         frame = self.bridge.imgmsg_to_cv2(msg, 'bgr')
35         results = self.model(frame)
36
37         for result in results:
38             boxes = result.boxes
39             for box in boxes:
40                 x1, y1, x2, y2 = map(int, box.xyxy[0])
41                 class_id = int(box.cls[0])
42                 label = self.model.names[class_id]
43
44                 center_x = int((x1 + x2) / 2) # Get center of bounding box
45                 center_y = int((y1 + y2) / 2)
46
47                 depth = self.get_depth(center_x, center_y) # Get depth at the center
48
49                 cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2) # Draw bounding box
50
51                 text = f"{label} {depth:.2f}m" # Display label + depth
52                 cv2.putText(frame, text, (x1, y1 - 10), # Draw label
53                             cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
54
55                 self.get_logger().info(f"Label: {label} at ({center_x}, {center_y}) - Depth: {depth:.2f}m")
56
57         if self.click_point is not None and self.depth_image is not None: # Show depth at click point if any
58             x, y = self.click_point
59             cv2.circle(frame, (x, y), 5, (0, 0, 255), -1)
60             distance = self.get_distance(x, y)
61             cv2.putText(frame, f"{distance:.2f}m", (x + 10, y - 10),
```

```

56 if self.click_point is not None and self.depth_image is not None: # Draw depth of clicked point in step
57     x, y = self.click_point
58     cv2.circle(frame, (x, y), 5, (0, 0, 255), 1)
59     distance = self.get_depth(x, y)
60     cv2.putText(frame, f"(distance: {distance})", (x + 10, y + 10),
61                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)
62
63     cv2.imshow("YOLO Detection", frame) # Display the frame
64     cv2.waitKey()
65
66 def depth_callback(self, msg):
67     self.depth_image = self.bridge.imgmsg_to_cv2(msg, "32SC1") # Convert depth image to CV2 format (32-bit unsigned int), depth in mm
68
69 def get_depth(self, x, y):
70     if self.depth_image is not None:
71         if 0 <= x < self.depth_image.shape[1] and 0 <= y < self.depth_image.shape[0]:
72             depth = self.depth_image[y, x]
73             if depth > 0:
74                 return depth / 255.0 # Convert mm to meters
75         return 0.0
76
77 def main():
78     rclpy.init()
79     node = RealSenseYOLO()
80     rclpy.spin(node)
81     node.destroy_node()
82     rclpy.shutdown()
83     cv2.destroyAllWindows()
84
85 if __name__ == '__main__':
86     main()

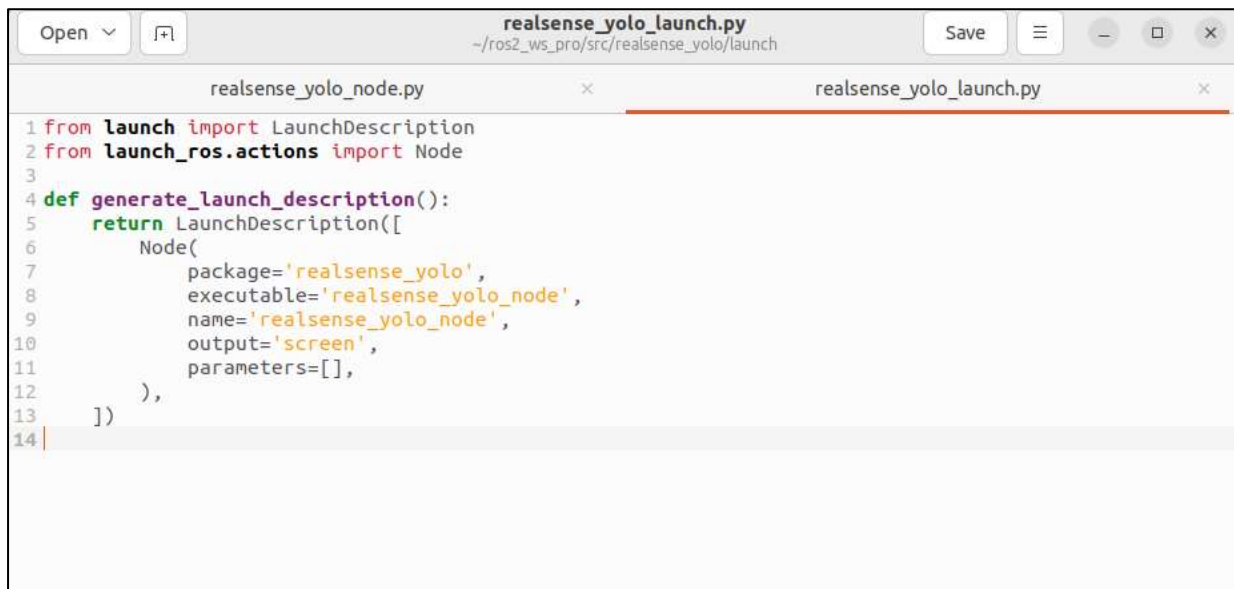
```

Fig 8. Python Script - *realsense_yolo_node.py*

5.4 Writing the Launch File and Running the Node

To simplify the execution of the object detection and depth estimation node, a custom launch file named *realsense_yolo_launch.py* was created inside the *realsense_yolo/launch/* directory.

Launch files in ROS 2 allow you to start nodes with specific parameters, configurations, or in combination with other nodes. This helps avoid repeatedly typing long commands in the terminal.



```

1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5     return LaunchDescription([
6         Node(
7             package='realsense_yolo',
8             executable='realsense_yolo_node',
9             name='realsense_yolo_node',
10            output='screen',
11            parameters=[],
12        ),
13    ])
14

```

Fig 9. Launch file

Once the launch file is saved, you can run the node using the following command:

```
ros2 launch realsense_yolo realsense_yolo_launch.py
```

5.5 Running All Nodes and Launching YOLO Detection with Depth Visualization

The final step involves executing all necessary commands across separate terminal windows to ensure smooth parallel operation of each ROS 2 node. In one terminal, the Intel RealSense D435i camera was launched using the command:

```
ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true
```

This activated the RGB and depth streams along with the point cloud output. Simultaneously, a second terminal was used to open RViz2 using:

```
rviz2
```

Within RViz, the /camera/depth/color/points topic was selected and visualized to confirm the camera was actively publishing 3D spatial data.

In a third terminal, the custom object detection node was launched using:

```
ros2 launch realsense_yolo realsense_yolo_launch.py
```

This command initialized the realsense_yolo_node, which performed real-time object detection using the YOLOv8 model on the RGB image stream. The script also accessed the aligned depth image to extract and calculate the distance of each detected object from the camera. Upon successful execution, an OpenCV display window popped up automatically, showing the live RGB feed with bounding boxes, object class labels, and depth values in meters overlaid on each detected object.

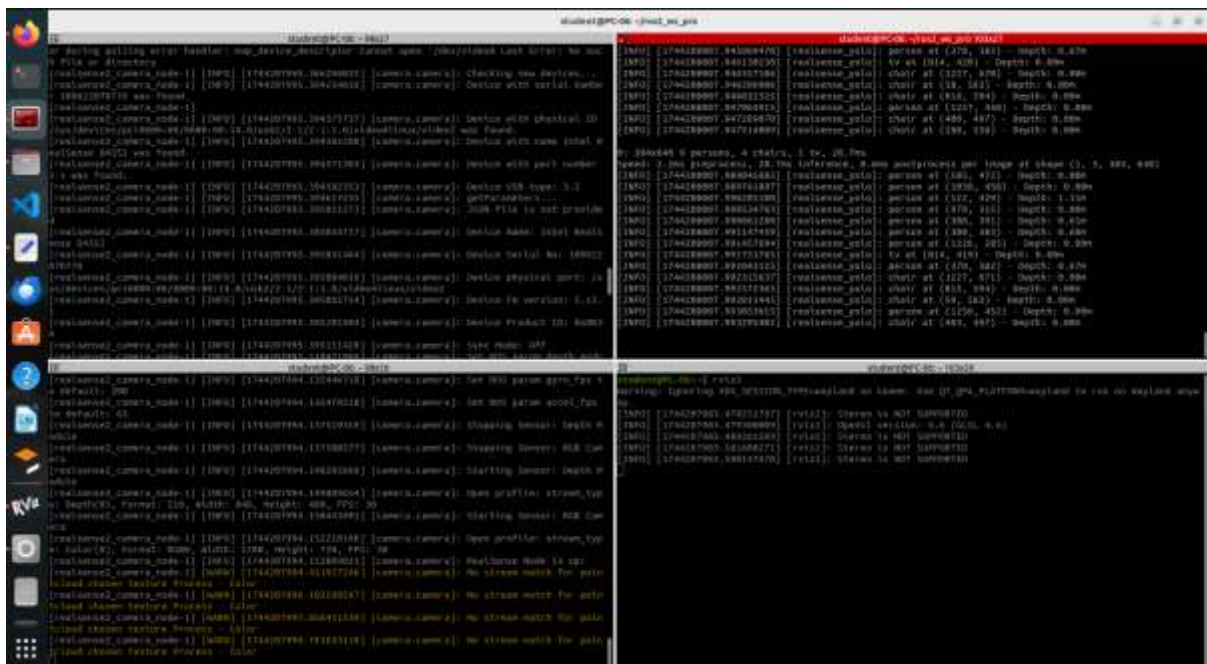


Fig 10. Terminal Outputs

VI. RESULTS AND ANALYSIS

The integrated object detection and depth estimation system was successfully implemented and tested in an indoor environment using the Intel RealSense D435i RGB-D camera. The system was evaluated based on its ability to detect multiple objects in real time and estimate their distance from the camera accurately.

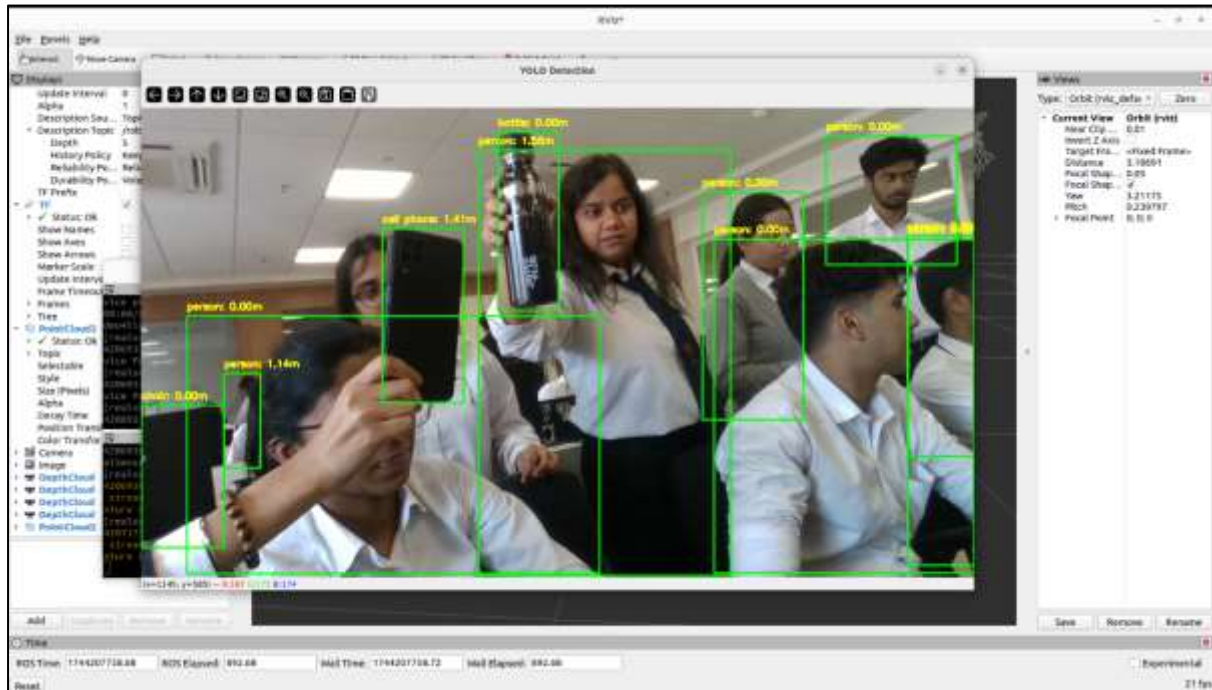


Fig 11. Result showing cell phone, person, and bottle

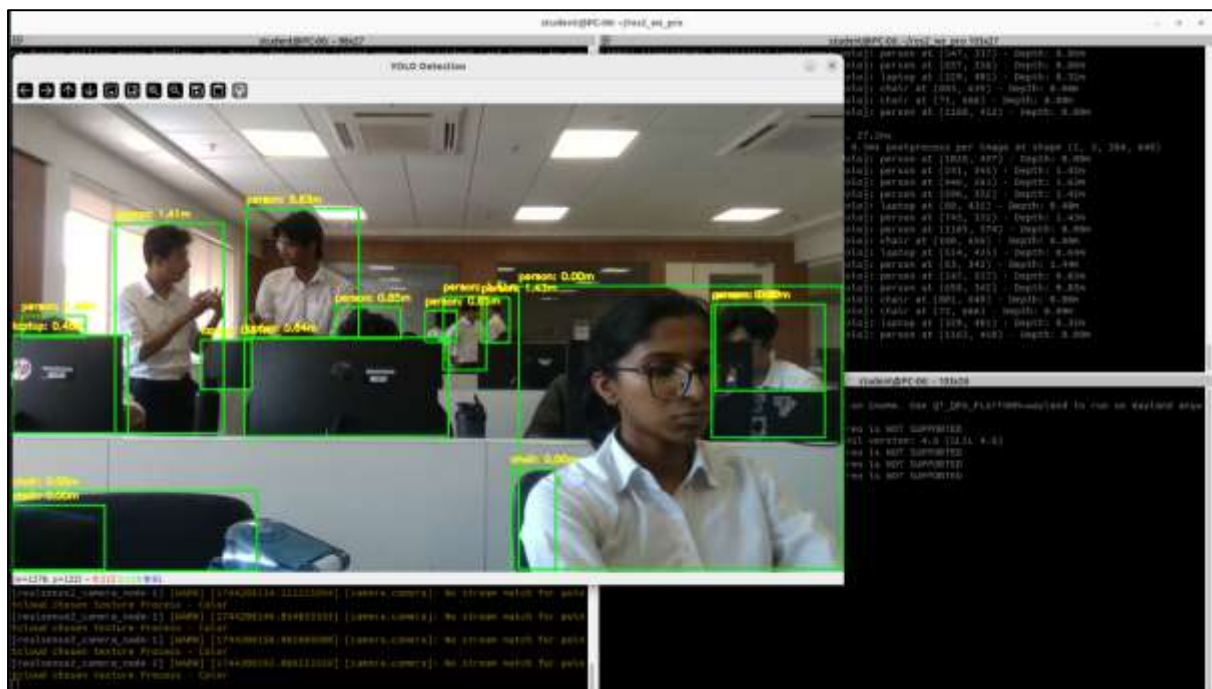


Fig 12. Result showing person and laptop and terminal indicating the depth values

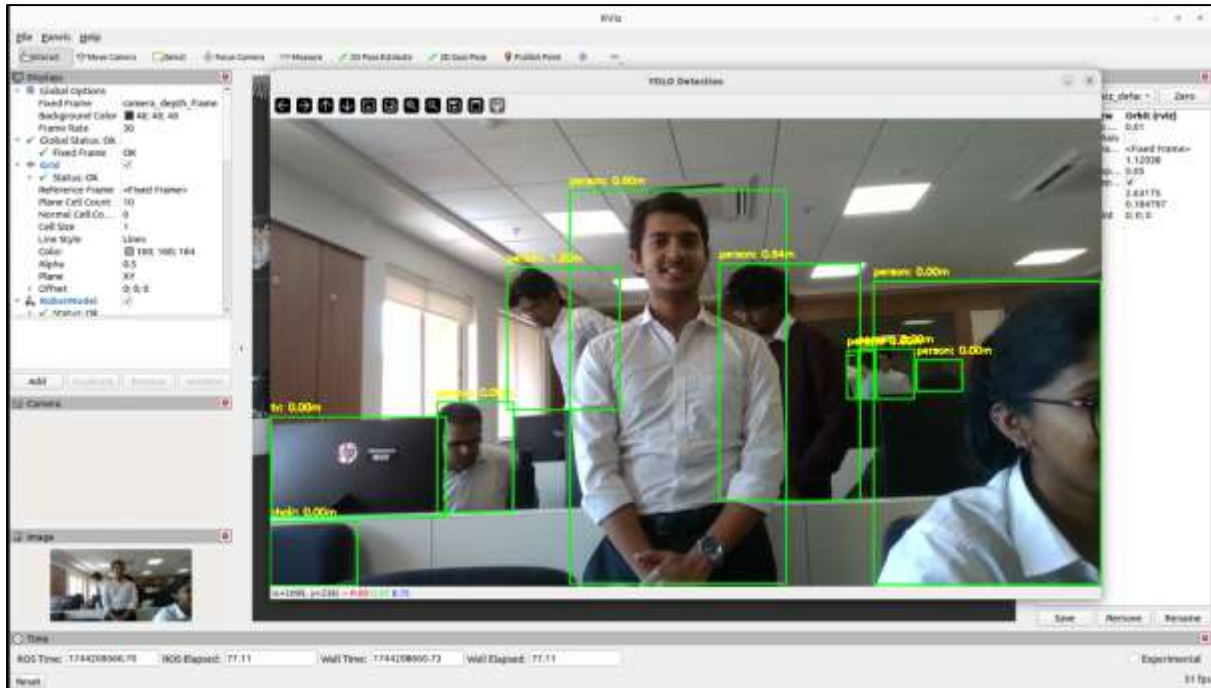


Fig 13. Result showing person

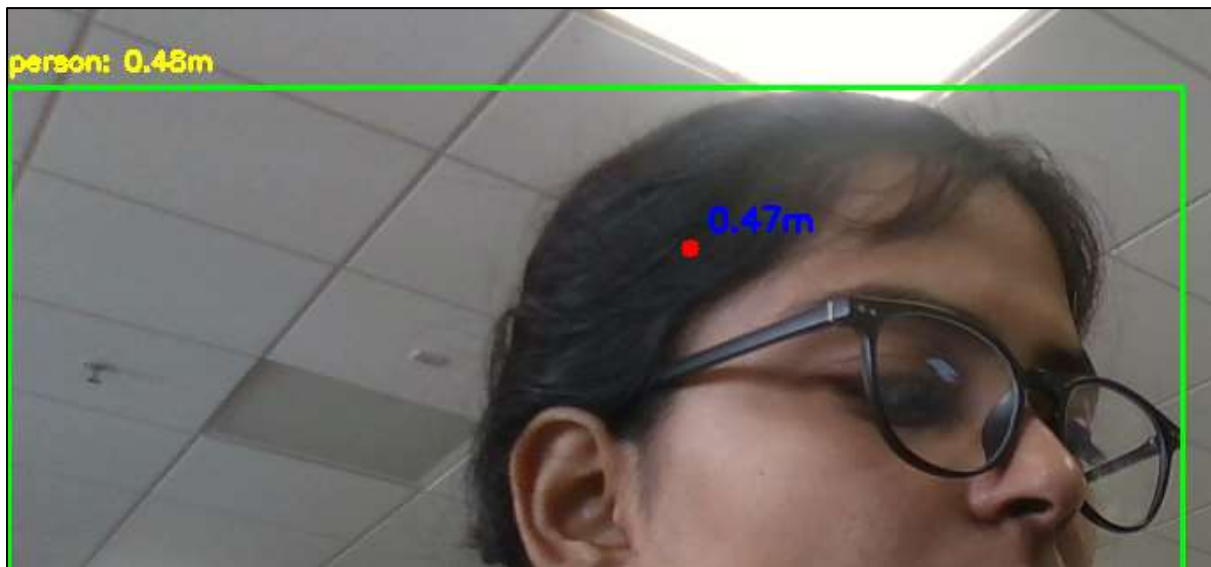


Fig 14. Result showing person and red dot in the image represents the user-selected (clicked) point on the image.

The YOLOv8 model was utilized to detect several common objects like people, chairs, TVs, and bottles in real-time using the RGB stream of the D435i camera. The object detection node was able to identify multiple objects in the frame at once and draw bounding boxes with correct labels. The Intel RealSense D435i provided depth data through the /camera/depth/image_rect_raw topic. This data was processed to obtain depth values in meters.

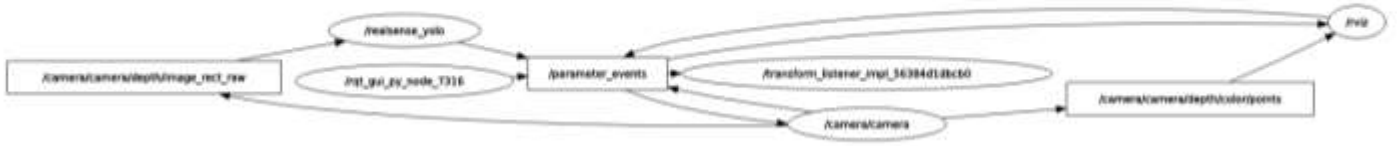


Fig 15. RQT Graph

VII. CONCLUSION AND FUTURE SCOPE

This project was able to successfully integrate object detection and depth estimation with the Intel RealSense D435i camera in a ROS 2 setup. With the integration of real-time RGB-D data, the YOLOv8 object detection model, and OpenCV processing, the system was capable of detecting and labeling objects in live camera feeds and measuring their distance from the camera with precision using depth data. The employment of ROS 2 permitted modular communication among nodes, and RViz2 gave the capability of a strong tool for visualizing the 3D scene as well as validating sensor output.

During the development process, critical functionalities like topic synchronization, coordinate translation, and visualization of data were managed efficiently through conventional ROS 2 practices. The PointCloud2 information from the RealSense sensor also improved spatial perception and offered a complete 3D scene of the environment. The configuration forms a sound basis for exploring robotic applications like autonomous navigation, obstacle detection, and environment mapping.

In conclusion, the project succeeded in its desired goals and validated the possibility of creating a practical, real-time visual perception system with low-cost hardware and open-source software. The learned concepts and structure here can further be extended to more advanced robotic processes involving spatial perception and real-time decision-making.

This project lays the groundwork for a wide range of advanced applications. In future work, the system can be enhanced by integrating it with a mobile robot platform for autonomous navigation and obstacle avoidance. Using a GPU or embedded system like the NVIDIA Jetson series could significantly improve the real-time performance of the YOLOv8 model. The system can also be extended to support SLAM (Simultaneous Localization and Mapping), dynamic object tracking, or even multi-sensor fusion for higher accuracy. Additionally, implementing a web-based interface or adding voice/gesture interaction could make the system more user-friendly for smart environments and assistive robotics.

REFERENCES

- [1] J. T. M. R. Sukkpranhachai Gatesichapakorn, “ROS based Autonomous Mobile Robot Navigation using 2D LiDAR and RGB-D Camera,” *2019 First International Symposium on Instrumentation, Control, Artificial Intelligence, and Robotics (ICA-SYMP)*.
- [2] Q. H. H. L. Dong XU, “Object detection on robot operation system,” *2016 IEEE 11th Conference on Industrial Electronics and Applications (ICIEA)*, 2016.
- [3] K. M. a. G. F. Sergio Trejo, “Depth map estimation methodology for detecting free-obstacle navigation areas,” *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2019.
- [4] D. Chikurtev, “Vision system for recognizing objects using Open Source Computer Vision (OpenCV) and Robot Operating System (ROS),” *Institute of Information and Communication Technologies – BAS*.

<https://www.intelrealsense.com/sdk-2/>

<https://dev.intelrealsense.com/docs/ros2-wrapper>

<https://github.com/IntelRealSense/realsense-ros>

<https://index.ros.org/p/rviz2/github-ros2-rviz2/>

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Understanding-Tf2.html>